

Tópicos de IA

Zuriel Dathan Mora Felix

Unidad 2

N reinas mediante búsqueda tabú

Archuleta Izabal Luis Enrique (22170561)

Ramirez Ruiz Santiago (22170783)

09-10-2026

Índice

1.-Descripcion del problema 3
2.-Representación de P y S: 4
3.-Diagrama de flujo: 5
4.-Implementación en Python: 6
5.-Repositorio 13
6.-Referencias 14

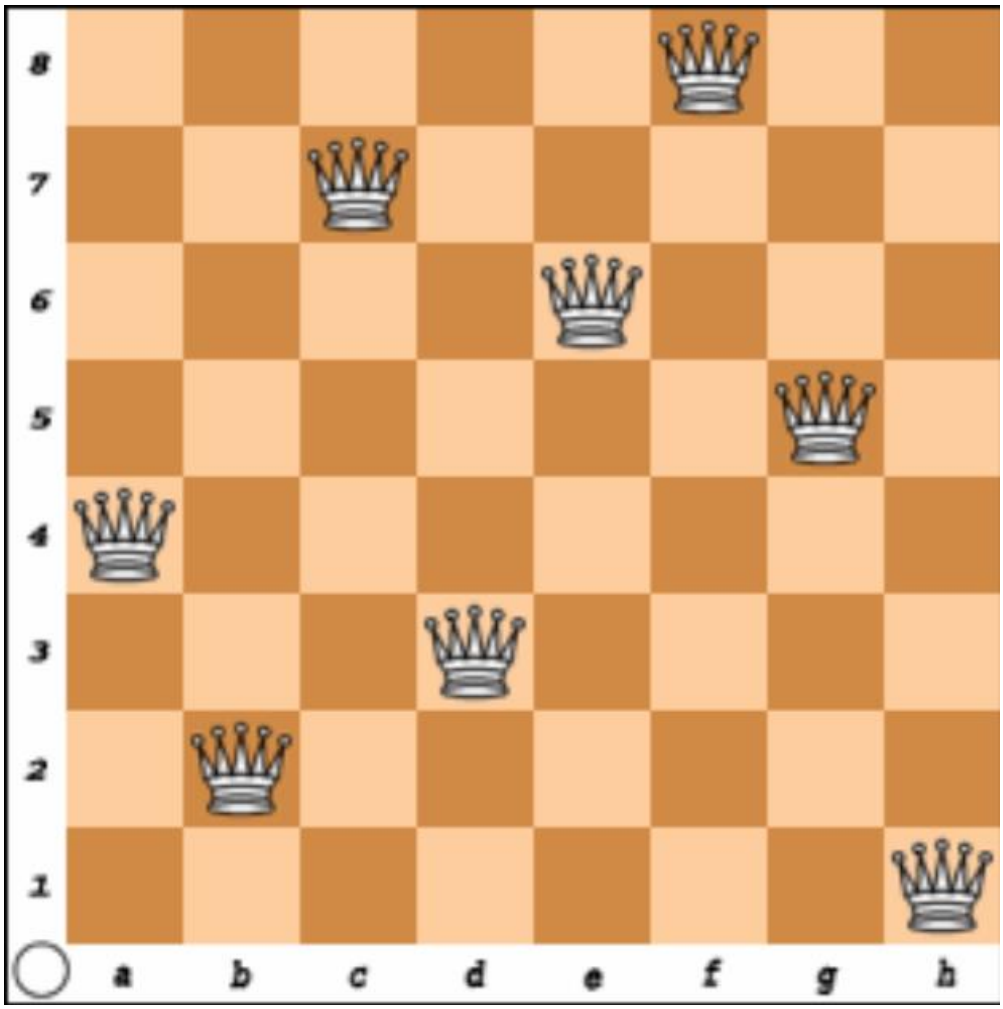
1.-Descripcion del problema

Problema de las N reinas

Este problema consiste en encontrar una distribución de n reinas en un tablero NxN de tal forma que no colisionen. Esto quiere decir que no pueden encontrarse dos reinas en la misma fila, columna o diagonal.

Ejemplo (N=8):

El problema original está hecho para un tablero de 8x8. Para N= 8, existen 92 soluciones posibles. Visualizándose de la siguiente forma:



2.-Representación de P y S:

En la búsqueda tabú la P representa la población o espacio de búsqueda y S la representación de un estado o solución individual del espacio.

Representación de S:

Para representar una posible solución (S) se puede utilizar con un arreglo o vector unidimensional donde el índice represente la fila y el valor almacenado represente la posición de la reina.

S=[r0, r1, r3, ... , r N-1]

Donde r es un valor entero entre 0 y N-1

Ejemplo para N=8:

[6,3,5,7,1,4,2,8]

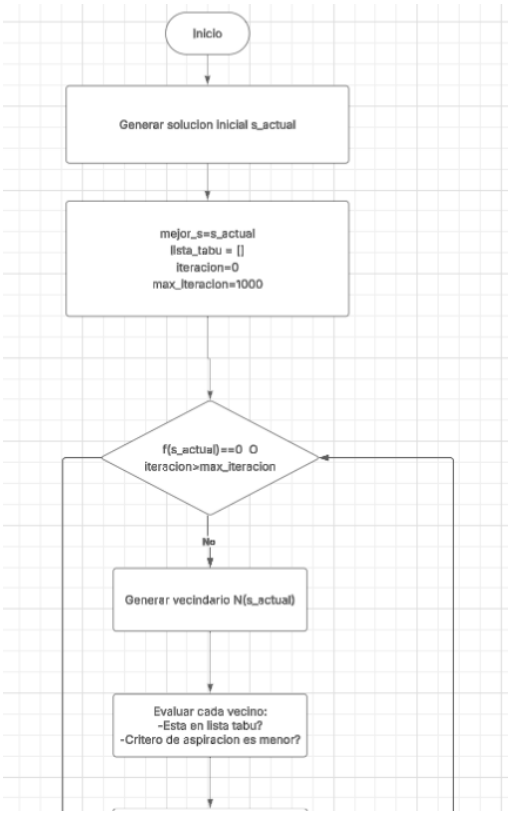
Representación de P:

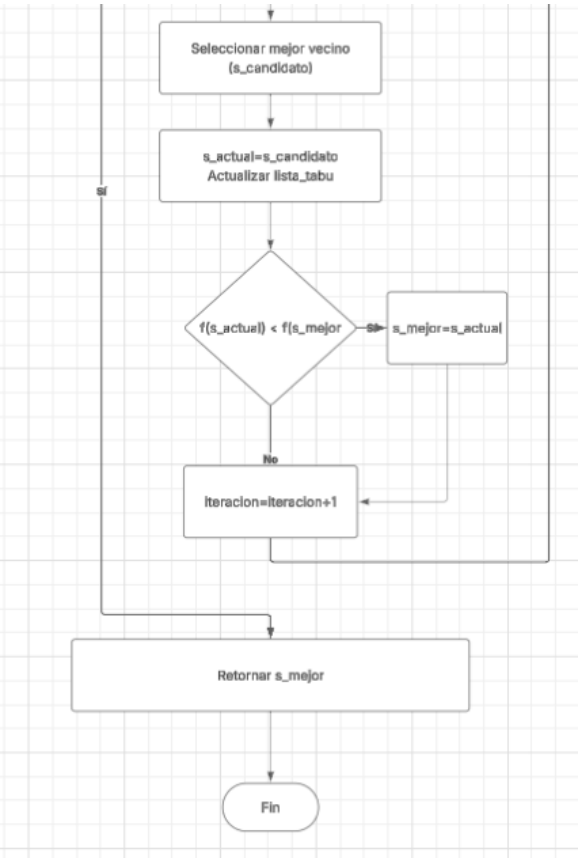
Con la solución implementada, P representa el espacio de búsqueda (el conjunto de todas las configuraciones posibles del tablero que el algoritmo puede explorar).

P está definido matemáticamente como el **Grupo Simétrico SN** (el conjunto de todas las permutaciones posibles de tamaño N).

$$P=SN =\{\text{Todas las permutaciones de } 0,1,2,\ldots,N-1)\}$$

3.-Diagrama de flujo:





4.-Implementación en Python:

```
def calcular_colisiones(s):

    n = len(s)
    colisiones = 0
    for i in range(n):
        for j in range(i + 1, n):
            # Condición de ataque diagonal: |s[i] - s[j]| == |i - j|
            if abs(s[i] - s[j]) == (j - i):
                colisiones += 1
    return colisiones
```

Para calcular la colisión diagonal se utiliza la siguiente formula:

Distancia vertical |s[i] – s[j]| == distancia horizontal (j-i)

Donde s[i], s[j] son las filas de las dos reinas, que se compara e i,j son las columnas.

Ejemplo:

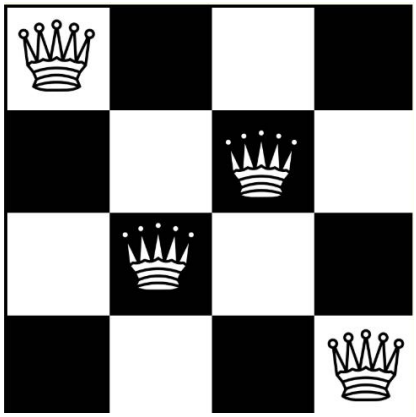
S=[0,2,1,3]

Reyna 0:

Columna i = 0 Fila s[i]= 0

Reyna 3:

Columna j=3 Fila s[j]=3



Distancia vertical = |0-3|=**3** Distancia horizontal =|0-3|=**3**

Como la distancia vertical y la horizontal son la misma existe colisión, como se ve en la imagen.

```
def resolver_n_reinas_tabu(n, max_iteraciones=1000, tenencia_tabu=10):
```

Primeramente, inicializamos el momento en que comienza a ejecutarse el algoritmo para tomar el tiempo total:

```
def resolver_n_reinas_tabu(n, max_iteraciones=1000, tenencia_tabu=10):
inicio_tiempo = time.perf_counter()
```

Se crea una lista de n elementos ordenados ejemplo s=[0,1,2,3,4,5,6,7] en el cual cada columna es el índice y el valor la fila, después con random.shuffle se mezclan los elementos al azar mediante el algoritmo Fisher-Yates el cual consiste en desordenar un arreglo de forma aleatoria y uniforme.

```
s_actual = list(range(n))
    random.shuffle(s_actual)
```

Por ultimo se asigna la solución inicial a la mejor solución y sus colisiones, se inicializa lista tabú, numero de iteración y movimientos realizados.

```
s_mejor = list(s_actual)
colisiones_s_mejor = calcular_colisiones(s_mejor)
colisiones_s_actual = colisiones_s_mejor
lista_tabu = []
iteracion = 0
```

```
movimientos_realizados = 0
```

Para que el método tabú finalice debe encontrarse la solución sin colisiones o que se alcance el número máximo de iteraciones. Después se inicializan el mejor vecino del vecindario y sus colisiones el cual es un numero positivo inifinito para que pueda compararse además del movimiento elegido el cual es el swap.

```
while colisiones_s_actual > 0 and iteracion < max_iteraciones:
        mejor_vecino = None

        colisiones_mejor_vecino = float('inf')

        movimiento_elegido = None
```

Para aplicar el tabu, utilizamos una doble iteración i,j las cuales son dos columnas diferentes del tablero.

```
        for i in range(n):
            for j in range(i + 1, n):
```

Despues se crea una copia de la solución actual para que se pueda alterar, haciendo el swap entre las columnas

```
s_vecino = list(s_actual)
s_vecino[i], s_vecino[j] = s_vecino[j], s_vecino[i] #swap
```

Se calculan las colisiones del vecino, su swap y su swap inverso es decir que si era [1,2,3,4] Y se hizo swap [2,1,3,4] se guardaran (1,2) y (2,1).

```
colisiones_vecino = calcular_colisiones(s_vecino)

movimiento = (s_actual[i], s_actual[j])

movimiento_inverso = (s_actual[j], s_actual[i])
```

Después se comprueba si el swap no esta dentro de la lista tabú.

```
es_tabu = (movimiento in lista_tabu) or (movimiento_inverso in lista_tabu)
```

Por ultimo comprueba que no esté en la lista tabú y evalua el criterio de aspiración (que la solución actual sea mejor que la mejor histórica) después compara al mejor vecino del vecindario para guardar la solución y sus colisiones.

```
if not es_tabu or colisiones_vecino < colisiones_s_mejor:
    if colisiones_vecino < colisiones_mejor_vecino:
         colisiones_mejor_vecino = colisiones_vecino
```

```
            mejor_vecino = s_vecino
            movimiento_elegido = movimiento
```

Después se valida que se haya encontrado a un mejor vecino si no sale del for

```
if mejor_vecino is None:
            break
```

Al final se asigna el mejor vecino del vecindario como la mejor solución actual con sus colisiones y se aumenta en uno el numero de movimientos realizados.

```
    s_actual = mejor_vecino

        colisiones_s_actual = colisiones_mejor_vecino
        movimientos_realizados += 1
```

Despues se agrega el swap a la lista tabú y se comprueba que la lista no este llena, si lo esta se libera mediante FIFO.

```
 lista_tabu.append(movimiento_elegido)
        if len(lista_tabu) > tenencia_tabu:
            lista_tabu.pop(0) # FIFO
```

Por último si la solución actual es mejor que la mejor solución se asigna como la nueva mejor solución y se aumenta el contador de colisiones.

```
if colisiones_s_actual < colisiones_s_mejor:
            s_mejor = list(s_actual)
            colisiones_s_mejor = colisiones_s_actual
iteracion += 1
```

Al terminar el algoritmo se calcula el tiempo total que tardo en funcionar.

```
    fin_tiempo = time.perf_counter()
    tiempo_ejecucion = fin_tiempo - inicio_tiempo
```

Se creo una función para representar el tablero, esta toma la mejor solución que es un vector de n números, donde el índice es la columna y su valor la fila, esta función recorre este vector e imprime R cuando detecta que la columna y la fila son las mismas y . cuando no.

```
def imprimir_tablero(s):
    n = len(s)
    print("\n--- Tablero Resultante ---")
    for fila in range(n):
        linea = ""
        for col in range(n):
            if s[col] == fila:
                linea += " R "
            else:
                linea += " . "
        print(linea)
```

Por ultimo se hace la ejecución del método principal el cual inicializa los parámetros del numero de reinas, máximo de iteraciones y la tenencia tabú, llamando al método resolver_n_reinas e imprimiendo las métricas establecidas las cuales son la solución encontrada, el numero de colisiones que tiene esta solución, iteraciones realizadas, los movimientos y el tiempo de ejecución. Por ultimo se imprime el tablero de la mejor solución.

```
if __name__ == "__main__":
    N = 8  # Tamaño del tablero
    MAX_ITER = 1000
    TENENCIA_TABU = 3

    print(f"Resolviendo {N}-Reinas con Búsqueda Tabú...")
    resultado = resolver_n_reinas_tabu(n=N, max_iteraciones=MAX_ITER,
tenencia_tabu=TENENCIA_TABU)

    # Métricas e información final
    print("\n=== MÉTRICAS DE EJECUCIÓN ===")
    print(f"Solución encontrada: {resultado['s_mejor']}")
    print(f"Colisiones restantes: {resultado[colisiones]} (0 = Éxito
total)")
    print(f"Iteraciones ejecutadas: {resultado['iteraciones']}")
    print(f"Movimientos realizados: {resultado['movimientos']}")
    print(f"Tiempo de ejecución: {resultado['tiempo_segundos'] * 1000:.3f}
ms ({resultado['tiempo_segundos']:.6f} s)")

    if resultado[colisiones] == 0:
        imprimir_tablero(resultado['s_mejor'])
    else:
        print("\nNo se encontró una solución óptima en el límite de
iteraciones.")
```

Código completo:

```
import random
import time

def calcular_colisiones(s):

    n = len(s)
    colisiones = 0
    for i in range(n):
        for j in range(i + 1, n):
            # Condición de ataque diagonal: |s[i] - s[j]| == |i - j|
```

```
            if abs(s[i] - s[j]) == (j - i):
                colisiones += 1
    return colisiones

def resolver_n_reinas_tabu(n, max_iteraciones=1000, tenencia_tabu=10):
    inicio_tiempo = time.perf_counter()

    # Generar solución inicial con permutacion aleatoria
    s_actual = list(range(n))
    random.shuffle(s_actual)

    # Inicialización de variables
    s_mejor = list(s_actual)
    colisiones_s_mejor = calcular_colisiones(s_mejor)
    colisiones_s_actual = colisiones_s_mejor
    lista_tabu = []
    iteracion = 0
    movimientos_realizados = 0

    # Bucle principal de optimización
    while colisiones_s_actual > 0 and iteracion < max_iteraciones:
        mejor_vecino = None
        colisiones_mejor_vecino = float('inf')
        movimiento_elegido = None

        # Generar y evaluar el vecindario completo N(s_actual) mediante
swaps
        for i in range(n):
            for j in range(i + 1, n):
                # Generar vecino aplicando Swap entre columnas(i, j)
                s_vecino = list(s_actual)
                s_vecino[i], s_vecino[j] = s_vecino[j], s_vecino[i]

                colisiones_vecino = calcular_colisiones(s_vecino)
                movimiento = (s_actual[i], s_actual[j]) # Atributo del
movimiento
                movimiento_inverso = (s_actual[j], s_actual[i])

                es_tabu = (movimiento in lista_tabu) or
(movimiento_inverso in lista_tabu)

                if not es_tabu or colisiones_vecino < colisiones_s_mejor:
                    if colisiones_vecino < colisiones_mejor_vecino:
                        colisiones_mejor_vecino = colisiones_vecino
                        mejor_vecino = s_vecino
```

```
                        movimiento_elegido = movimiento

        # Si no hay movimientos válidos posibles
        if mejor_vecino is None:
            break

        #Transición al mejor vecino seleccionado
        s_actual = mejor_vecino
        colisiones_s_actual = colisiones_mejor_vecino
        movimientos_realizados += 1

        # Actualizar Lista Tabú
        lista_tabu.append(movimiento_elegido)
        if len(lista_tabu) > tenencia_tabu:
            lista_tabu.pop(0) # FIFO

        # Actualizar mejor solución global si aplica
        if colisiones_s_actual < colisiones_s_mejor:
            s_mejor = list(s_actual)
            colisiones_s_mejor = colisiones_s_actual

        iteracion += 1

    fin_tiempo = time.perf_counter()
    tiempo_ejecucion = fin_tiempo - inicio_tiempo

    return {
        "s_mejor": s_mejor,
        "colisiones": colisiones_s_mejor,
        "iteraciones": iteracion,
        "movimientos": movimientos_realizados,
        "tiempo_segundos": tiempo_ejecucion
    }

def imprimir_tablero(s):
    n = len(s)
    print("\n--- Tablero Resultante ---")
    for fila in range(n):
        linea = ""
        for col in range(n):
            if s[col] == fila:
                linea += " R "
            else:
                linea += " . "
        print(linea)
```

```
# EJECUCIÓN DEL ALGORITMO

if __name__ == "__main__":
    N = 8  # Tamaño del tablero
    MAX_ITER = 1000
    TENENCIA_TABU = 3

    print(f"Resolviendo {N}-Reinas con Búsqueda Tabú...")
    resultado = resolver_n_reinas_tabu(n=N, max_iteraciones=MAX_ITER,
tenencia_tabu=TENENCIA_TABU)

    # Métricas e información final
    print("\n=== MÉTRICAS DE EJECUCIÓN ===")
    print(f"Solución encontrada: {resultado['s_mejor']}")
    print(f"Colisiones restantes: {resultado['colisiones']} (0 = Éxito
total)")
    print(f"Iteraciones ejecutadas: {resultado['iteraciones']}")
    print(f"Movimientos realizados: {resultado['movimientos']}")
    print(f"Tiempo de ejecución: {resultado['tiempo_segundos'] * 1000:.3f}
ms ({resultado['tiempo_segundos']:.6f} s)")

    if resultado['colisiones'] == 0:
        imprimir_tablero(resultado['s_mejor'])
    else:
        print("\nNo se encontró una solución óptima en el límite de
iteraciones.")
```

5.-Repositorio

Para ejecutar el programa presentado debe descargarse en el siguiente repositorio: https://github.com/LuisArchuleta/Topicos-IA

El archivo se encuentra en Modulo 2/ N reinas/ nreinas.py

6.-Referencias

Fernández, I. (2025, 18 febrero). *Problema de las N reinas - MoraCoCR*. MoraCoCR. https://www.mora.co.cr/articulos/ingenieria-en-computacion/analisis-de-algoritmos/problema-de-las-n-reinas

Búsqueda tabú. (s. f.). https://jonasvelasco.com/algoritmos/7/bl_cuadricula_tabu_search

Azad, T. (2025, 9 mayo). *Fisher-Yates Shuffle: The Right Way to Randomize an Array*. DEV Community. https://dev.to/tanvir_azad/fisher-yates-shuffle-the-right-way-to-randomize-an-array-4d2p